

LISTADO DE TAREAS A EJECUTAR

Tareas Middleware:

1. Integrar RabbitMQ.
2. Crear interfaz para emisión basada en grupos.

Tareas Client:

1. Generar archivos de `transactions.csv` para cada cliente con un total de 500000 transacciones por cliente.
2. Implementar esqueleto usando el de TP-Coordinación como base y adecuar al negocio.

Tareas protocolo TCP/IP:

1. Mensaje de TRAN_RECORD hacia Gateway
[id_client, id_from_bank, from_account, id_to_bank, to_account, currency, amount, payment_format]
Nota: amount corresponde a la columna "Amount Paid" del dataset.
2. Mensaje de CLIENT_EOF hacia Gateway
[id_client, EOF]
3. Mensaje de Q1_RESULT hacia Client
[id_client, id_from_bank, from_account, id_to_bank, to_account, amount]
4. Mensaje de Q2_RESULT hacia Client
[id_client, id_from_bank, from_account, bank_name, amount]
Nota: para obtener bank_name debemos coordinar el acceso a un archivo que contiene esa información.
5. Mensaje de Q3_RESULT hacia Client
[id_client, id_from_bank, from_account, payment_format, amount]
6. Mensaje de Q4_RESULT hacia Client
[id_client, id_from_bank, from_account]
7. Mensaje de Q5_RESULT hacia Client
[id_client, accumulator_result]

Tareas Gateway:

1. Implementar esqueleto usando el de TP-Coordinación como base y adecuar al negocio.
2. Implementar objeto Parser que sea capaz de generar un UUID por cliente y parsear los mensajes "TRAN_RECORD" y "CLIENT_EOF" en una estructura de datos.
3. Implementar lógica para que el Gateway "se entere" cuando termina la ejecución de cada query, y así pueda avisarle al cliente.

Tareas Processing Node:

1. Implementar esqueleto usando el de TP-Coordinación como base.
2. Recibir mensajes enviados a través de la cola del middleware.
Los mensajes son TRAN_RECORD o CLIENT_EOF.
3. Asegurar la concurrencia del procesamiento de cada query mediante *multithreading*.
Justificación uso de multithreading: en nuestro diseño, el procesamiento más pesado corresponde a operaciones I/O-bound (por llamada a API externa), lo que permite solapar tiempos de espera. Los procesamientos del resto de queries son breves y no

generan carga significativa de CPU, por lo que el impacto del GIL en Python es despreciable, evitando así el overhead de multiprocessing.

4. Implementar el procesamiento de cada query para cada transacción:

a. **Query 1:**

- i. Filtro:
 1. Debe ser transacción USD
 2. Debe ser con monto < 50
- ii. Enviar al Gateway mensaje Q1_RESULT.
- iii. Cuando llega CLIENT_EOF (y se terminaron de procesar las transacciones) avisar al Gateway que terminó la ejecución de la query 1 para el cliente específico.

b. **Query 2:**

- i. Filtro:
 1. Debe ser transacción USD
- ii. Tener un diccionario por id_client, que dentro tenga otro diccionario por la clave (id_bank, bank_name).
- iii. Con la nueva transacción, actualizar la máxima transacción para el cliente y el banco específicos.
- iv. Cuando llega CLIENT_EOF mandar el resultado al nodo de agregación a través de un exchange.
Ruteo: por par (id_client, id_bank). Calcular exchange destino con un hash determinístico.

Mensaje Q2_PARTIAL_BANK_MAX:

```
[query_num, id_client, id_bank, account, bank_name, amount]
```

- v. Mandar CLIENT_EOF a los nodos de agregación.
- vi. Limpiar diccionarios para el cliente específico.

c. **Query 3:**

- i. Filtro:
 1. Debe ser transacción USD
- ii. Tener un diccionario por id_client, que dentro tenga otro diccionario por la clave payment_format que guarde una tupla con (1) la información correspondiente a la suma de los montos de las transacciones que llegan para el periodo requerido, y (2) la cantidad de transacciones. Estos serán los valores requeridos para calcular el promedio.
- iii. Para transacciones en el periodo [2022-09-01, 2022-09-05]:
 1. Actualizar el diccionario por clave payment_format sumando el valor de los montos de las transacciones e incrementando el contador de transacciones.
- iv. Para transacciones en el periodo [2022-09-06, 2022-09-15]:
 1. Mandar la transacción al nodo de agregación.
Ruteo: por par (id_client, payment_format). Calcular exchange destino con un hash determinístico

Mensaje Q3_TRANSACTION:

```
[query_num, id_client, id_bank, account, payment_format, amount]
```

- v. Cuando llegue CLIENT_EOF mandar al nodo de agregación los valores almacenados en el diccionario para calcular el promedio.
Ruteo: por par (id_client, payment_format). Calcular exchange destino con un hash determinístico

Mensaje Q3_PARTIAL_AVG:

```
[query_num, id_client, payment_format, sum_amount, counter_tran]
```

- vi. Mandar CLIENT_EOF a los nodos de agregación.
 - vii. Limpiar diccionarios para el cliente específico.
- d. **Query 4:**
- i. Filtro:
 - 1. Debe ser transacción USD
 - 2. Debe ser del periodo [2022-09-01, 2022-09-05]
 - ii. Para cada transacción ($A \rightarrow X_1$) emito Transaction[From A, To X_1]
 - iii. Envío la transacción a través de los exchanges hacia los nodos de agregación.
Ruteo: se generan 2 keys, una por la cuenta origen (A) y otra por la cuenta destino (X_1). Calcular exchange destino con hashes determinísticos.

Mensaje Q4_TRAN_2ACC:

```
[query_num, id_client, id_from_bank, from_account, id_to_bank, to_account]
```

- iv. Mandar CLIENT_EOF a los nodos de agregación.
- e. **Query 5:**
- i. Filtro:
 - 1. Debe ser del periodo [2022-09-01, 2022-09-05]
 - 2. Formato de pago "Wire" o "ACH"
 - 3. Su monto (convertido a USD) debe ser menor a 1 (resolver comunicación con la API para la conversión)
 - ii. Se tiene un diccionario por id_client que mapea a un contador para las transacciones que cumplan con el filtro.
Incrementar contador cuando llegue una que lo cumpla.
 - iii. Cuando llegue CLIENT_EOF mandar al nodo de agregación el contador parcial.
Ruteo: por id_client. Calcular exchange destino con un hash determinístico

Mensaje Q5_PARTIAL_COUNTER:

```
[query_num, id_client, counter]
```

- iv. Mandar CLIENT_EOF a los nodos de agregación.
- v. Limpiar diccionarios para el cliente específico.

Tareas Aggregation Node:

1. Implementar esqueleto usando el de TP-Coordinación como base.
2. Recibir mensajes a través del exchange específico de cada nodo de agregación.
3. Asegurar la concurrencia del procesamiento de cada query mediante *multithreading*.
4. Implementar el procesamiento de cada query para cada transacción:

a. **Query 2:**

- i. En un mismo nodo de agregación llegan los cálculos parciales para un mismo banco de un mismo cliente (Q2_PARTIAL_BANK_MAX), que el nodo deberá almacenar.
- ii. Cuando le lleguen al nodo $N = \text{cantidad_nodos_de_procesamiento}$ mensajes CLIENT_EOF, el nodo deberá calcular el máximo entre los parciales.
- iii. Enviar al Gateway mensaje Q2_RESULT.
- iv. Limpiar diccionarios para el cliente específico.
- v. Avisar al Gateway que terminó la ejecución de la query 2 para el cliente específico.

b. **Query 3:**

- i. Cuando llega un mensaje Q3_TRANSACTION, almacenar la transacción en una colección por cliente y formato de pago.
- ii. Tener un diccionario por id_cliente, que dentro tenga otro diccionario por la clave payment_format que guarde los valores necesarios para resolver el promedio.
Cuando llega Q3_PARTIAL_AVG, actualizar los valores para ese formato de pago.
- iii. Cuando le lleguen $N = \text{cantidad_nodos_de_procesamiento}$ mensajes CLIENT_EOF, debe:
 1. Calcular el promedio por formato de pago con los valores almacenados.
 2. Recorrer las transacciones almacenadas y quedarse con solo aquellas con monto menor a 1 centesimo del promedio (promedio/100) para el formato de pago específico.
- iv. Enviar al Gateway mensaje Q3_RESULT.
- v. Limpiar diccionarios para el cliente específico.
- vi. Avisar al Gateway que terminó la ejecución de la query 3 para el cliente específico.

c. **Query 4:**

- i. Con las transacciones obtenidas a través del mensaje Q4_TRAN_2ACC, gracias al enrutamiento realizado, al menos un nodo de agregación va a ser capaz de generar cadenas de 3 transacciones en caso de que las encuentre (ej. $A \rightarrow X_1 \rightarrow B$). Inevitablemente otro nodo va a:
 1. Generar otra cadena duplicada $A \rightarrow X_1 \rightarrow B$.
Ejemplo:
NodoAggr1: le llega $(A \rightarrow X_1), (X_1 \rightarrow B)$. Forma $A \rightarrow X_1 \rightarrow B$.
NodoAggr2: le llega $(A \rightarrow X_1), (X_1 \rightarrow B)$. Forma $A \rightarrow X_1 \rightarrow B$ (dup.).
En ese caso ambas cadenas se envían igualmente al nodo join.
 2. Quedarse con una transacción almacenada que se descartará recién cuando llegue CLIENT_EOF.
Ejemplo:
NodoAggr1: le llega $(A \rightarrow X_1), (A \rightarrow X_1)$ (lo ignora), $(X_1 \rightarrow B)$.
Forma $A \rightarrow X_1 \rightarrow B$.

NodoAggr2: le llega (X1 □ B). Se queda con ese valor almacenado hasta que lo descarta cuando llega CLIENT_EOF.

- ii. Una vez se generan las cadenas de 3 transacciones, mandar a los nodos join.

Ruteo: por par (cuenta origen (ej. A), cuenta destino (ej. B)).

Mensaje Q4_TRAN_3ACC:

```
[query_num, id_client, id_from_bank, from_account, id_mid_bank, mid_account, id_to_bank, to_account]
```

- iii. Mandar CLIENT_EOF a los nodos de agregación cuando llegue.
- d. **Query 5:**
- i. En un mismo nodo de agregación llegan los contadores parciales de cada cliente (Q5_PARTIAL_COUNTER). El nodo deberá ir incrementando el valor de un nuevo contador que sume todos los contadores que le van llegando.
 - ii. Cuando le lleguen al nodo N=cantidad_nodos_de_procesamiento mensajes CLIENT_EOF, el nodo ya tendrá el valor final del contador.
 - iii. Enviar al Gateway mensaje Q5_RESULT.
 - iv. Limpiar diccionarios para el cliente específico.
 - v. Avisar al Gateway que terminó la ejecución de la query 5 para el cliente específico.

Tareas Join Node:

1. Implementar esqueleto usando el de TP-Coordinación como base.
2. Recibir mensajes a través del exchange específico de cada nodo de join.
3. Asegurar la concurrencia del procesamiento de cada query mediante *multithreading*.
4. Implementar el procesamiento de cada query para cada transacción:

a. **Query 4:**

- i. Gracias al mensaje Q4_TRAN_3ACC y al enrutamiento, en un mismo nodo se juntan todas las cadenas de 3 transacciones con mismo origen y destino (sin importar por qué cuenta intermedia pasan). Cada vez que llega una transacción de este estilo incrementar un contador para el par (cuenta origen, cuenta destino). Mantener un diccionario por id_client para almacenar los contadores.
- ii. Si llega una cadena duplicada se descarta.
- iii. Si el contador llega a más de 5, enviar Q4_RESULT al Gateway.
- iv. Si llegan más cadenas con el mismo origen y destino, ignorar (ya que ya se mando el resultado de la query para esas cuentas).
- v. Cuando le lleguen N=cantidad_nodos_de_procesamiento mensajes CLIENT_EOF limpiar diccionarios para el cliente específico.
- vi. Avisar al Gateway que terminó la ejecución de la query 4 para el cliente específico.

Tareas docker:

1. Generar archivo de docker compose con múltiples containers para cada nodo. Definir sus variables de entorno de acuerdo a lo necesario para la resolución de las queries.